Phase 6 — Tracking & Behaviour Analysis (Implementation Guide)

> Status: 🔲 Planned — design spec, ready to implement once Phase 5 (Multi-Scale Detection, built on the Phase 4 TRC-gated fusion detector) is trained and producing stable detections. Prereq: ✅ Phase 2 (data) · ✅ Phase 3 (preprocessing & TRC) · Phase 4 (fusion)
>
> - Phase 5 (detection) in progress. This is **Layer 5** of the 7-layer TriNetra architecture. It sits **between the fused-modality detector (Phase 4/5) and hierarchical threat reasoning (Phase 7)**.

---

1. The One-Line Answer

> **Phase 6 turns a stream of per-frame detections into persistent identities ("tracks"), then derives behavioural features from each track's motion history — and it does this in a way that stays aware of TRC, so a track isn't lost or mis-identified just because the thermal camera had a bad frame.**

---

2. Where Phase 6 Sits (The Big Picture)

```
Phase 4/5 (Layers 3-4)              Phase 6 (Layer 5)                    Phase 7
(Layer 6)
Fusion + Detection          →       Tracking & Behaviour         →       Threat
Reasoning
(boxes, classes, scores,            (track IDs, trajectories,            (risk tiers
from
 per-frame, per-image)               behaviour flags, per-track)          track +
behaviour)
```

Detection answers "what is in this frame, and where?" — independently, frame by frame. Tracking answers the harder question: **"is the person in frame 214 the same person as the person in frame 200?"** Without that identity link, nothing downstream — loitering time, path analysis, restricted-zone dwell, risk escalation over time — is even definable. Threat reasoning (Phase 7) consumes *tracks*, not detections.

Why this matters for TriNetra specifically: in a single-modality system, a tracker only has to survive RGB occlusion or thermal crossover independently. In TriNetra, the tracker can lean on *whichever modality Phase 3's TRC says is reliable in that frame* — e.g. if a person walks through a shadow (RGB fails) but the thermal signature is clean (high TRC), the track should be recoverable using thermal-derived motion/appearance cues instead of being dropped and re-assigned a new ID.

---

3. Input Contract (What Phase 6 Receives)

From the Phase 5 detector, per frame `t`:

```python
detections_t = [
    {
        "bbox": [x, y, w, h],       # COCO-style, absolute pixels
        "score": 0.87,              # detection confidence
        "class_id": 1,               # e.g. person
        "trc": 0.76,                 # Phase 3 TRC for this frame (scalar, or trc_map)
        "modality_contrib": {...},   # optional: which modality's features dominated
                                      # this detection, from the fusion gate
    },
    ...
]
```

Design requirement: the detector's output schema must carry `trc` (and, if enabled later, `trc_map`) **forward** into every detection dict — Phase 5's `FusionDetector` already has this value available (it consumed it for gating), it just needs to be attached to the output rather than discarded after the forward pass. This is a **one-line addition** to Phase 5's inference wrapper and is the only coupling Phase 6 needs from Phase 5.

---

4. Choosing a Tracking Algorithm — Comparative Analysis

This is the central engineering decision of Phase 6. Below is a structured comparison of the standard multi-object tracking (MOT) algorithms, evaluated specifically against TriNetra's constraints: **real-time budget (Phase 9), frequent partial occlusion in surveillance footage, and the availability of a per-frame reliability signal (TRC) that a generic tracker doesn't have.**

Algorithm	Core idea	Strengths	Weaknesses	Fit for TriNetra
SORT	Kalman filter + IoU matching + Hungarian assignment. No appearance model.	Extremely fast, simple, no training needed.	Frequent ID switches under occlusion; discards low-confidence detections entirely.	Baseline only — too fragile for surveillance-grade tracking.
DeepSORT	SORT + a CNN re-identification (Re-ID) embedding for appearance matching.	Much better ID stability across occlusion; mature, widely used.	Needs a trained Re-ID model; adds latency; embedding quality degrades in low light — exactly where thermal should help but DeepSORT has no concept of a second modality.	Good fallback / appearance backbone, but not modality-aware out of the box.
ByteTrack	Associates **all** detections, including low-confidence ones (two-stage matching: high-conf first, then low-conf), using only motion (IoU) — no Re-ID network required.	State-of-the-art MOTA on MOT17/MOT20 with *no appearance model at all*; very fast; simple to implement; low-confidence boxes (often partially occluded true positives) are recovered instead of discarded.	Purely motion-based — can still switch IDs when two tracks cross paths with similar motion.	**Best starting point.** Matches TriNetra's real-time goal (Phase 9) and its low-confidence detections are often exactly the "TRC is uncertain" frames — a natural hook to bias matching.
BoT-SORT	ByteTrack's association logic + a lightweight Re-ID branch + camera motion compensation (CMC).	Combines ByteTrack's robustness with appearance re-identification; camera motion compensation helps	Heavier than pure ByteTrack; Re-ID branch still needs training/fine-tuning.	**Best long-term target** — the natural "Stage 2" upgrade once thermal-assisted Re-ID is validated.

Algorithm	Core idea	Strengths	Weaknesses	Fit for TriNetra
		if the camera isn't perfectly static.		
OC-SORT	Observation-centric re-formulation of Kalman filtering to reduce error accumulation during occlusion, no Re-ID.	Very strong at recovering tracks after long occlusion without appearance cues; simple.	Still single-modality motion only.	Worth an ablation against ByteTrack; occlusion-heavy CCTV footage is its sweet spot.
StrongSORT	DeepSORT lineage with modern Re-ID + matching refinements (e.g. AFLink, GSI).	Strong ID-F1 scores in benchmarks.	Heaviest of the group; overkill before ByteTrack is even validated.	Reference point for later ablations, not a v1 pick.

4.1 Recommendation (the decision to make and defend)

Ship ByteTrack first (Stage 1), upgrade to a TRC-gated BoT-SORT-style hybrid (Stage 2) once the baseline is measured.

Reasoning:

1. **ByteTrack needs no training.** It is pure Kalman-filter motion tracking plus a two-stage Hungarian matching step. This mirrors the project's own precedent set in Phase 3 (TRC is unsupervised/no-training) and Phase 4 (only *new* modules are trained) — get a working, low-risk baseline first.
2. **ByteTrack's core innovation is exactly what TriNetra needs.** Standard trackers throw away detections below a confidence threshold (e.g. 0.5). ByteTrack keeps them and tries to match them to existing tracks using motion alone. In TriNetra, a detection with **low confidence but high TRC-weighted thermal support** (e.g. someone partially in shadow, RGB struggling, thermal clean) is precisely the case ByteTrack's second-stage matching is designed to rescue.
3. **It's fast enough for Phase 9's real-time budget** — no CNN forward pass for Re-ID, so the tracking stage adds negligible latency versus the detector itself.
4. **BoT-SORT is the principled Stage-2 upgrade**, not a competing choice — it literally extends ByteTrack's association logic with an optional Re-ID branch, so migrating later is incremental, not a rewrite.

4.2 The trade-off table (what to say if asked "why not just X?")

Approach	Trained components	Pros	Cons
A. SORT only	None	Simplest possible	Too many ID switches for a security use case
B. ByteTrack (recommended v1)	None	Fast, robust to occlusion via low-conf recovery, no training risk	No appearance model — can swap IDs on crossing paths
C. DeepSORT	Re-ID CNN	Strong appearance matching	Re-ID model needs training data + doesn't know about thermal
D. TRC-gated BoT-SORT (v2 target)	Re-ID CNN (dual-modal)	Best of both — motion + appearance + modality awareness	Most engineering effort; needs Phase 4/5's fused features to reuse

5. Architecture — TriNetra's Tracking Module

5.1 Directory layout (fits the existing repo structure)

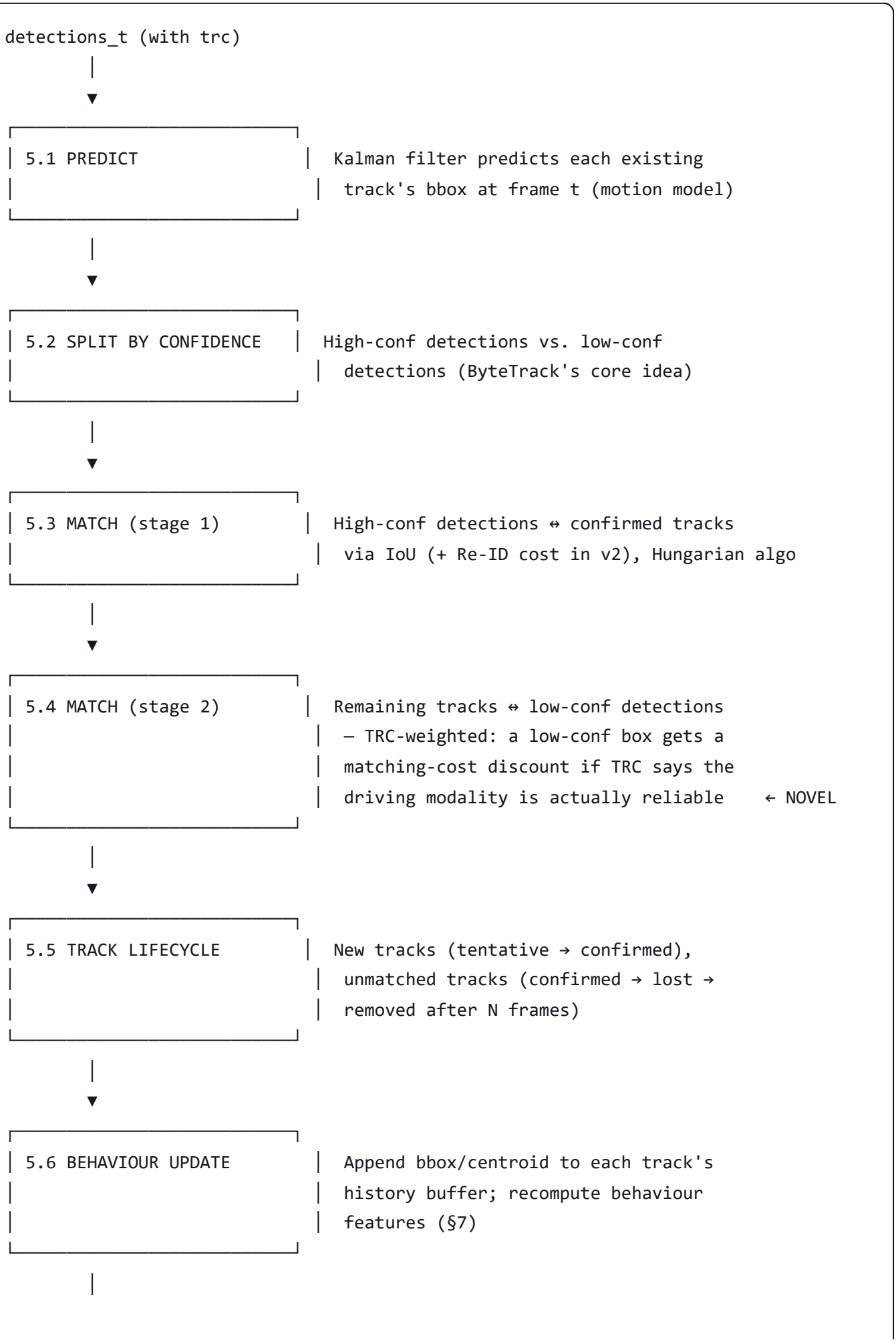
```
models/
└── tracker/
    ├── __init__.py
    ├── kalman_filter.py        # Constant-velocity Kalman filter for bbox state
    ├── byte_tracker.py         # ByteTrack: two-stage association (NEW, v1 core)
    ├── matching.py             # IoU cost, Hungarian assignment, gating
    ├── trc_gate.py             # TRC-aware association weighting (NOVEL — TriNetra-
specific)
    ├── reid_embedder.py        # Dual-modal Re-ID embedding (v2, BoT-SORT stage)
    ├── track.py                # Track object: state machine, history buffer
    └── botsort_tracker.py      # v2: ByteTrack + Re-ID + camera motion compensation

inference/
├── realtime.py                 # [MODIFY] wire tracker into the live loop
└── video.py                    # [MODIFY] wire tracker into offline processing

utils/
└── behaviour.py                # [NEW] trajectory feature extraction + anomaly
flags

configs/
└── default.yaml                # [MODIFY] add `tracking:` block
```

5.2 Core pipeline per frame



```
        ▼
Tracks with IDs + behaviour flags → Phase 7
```

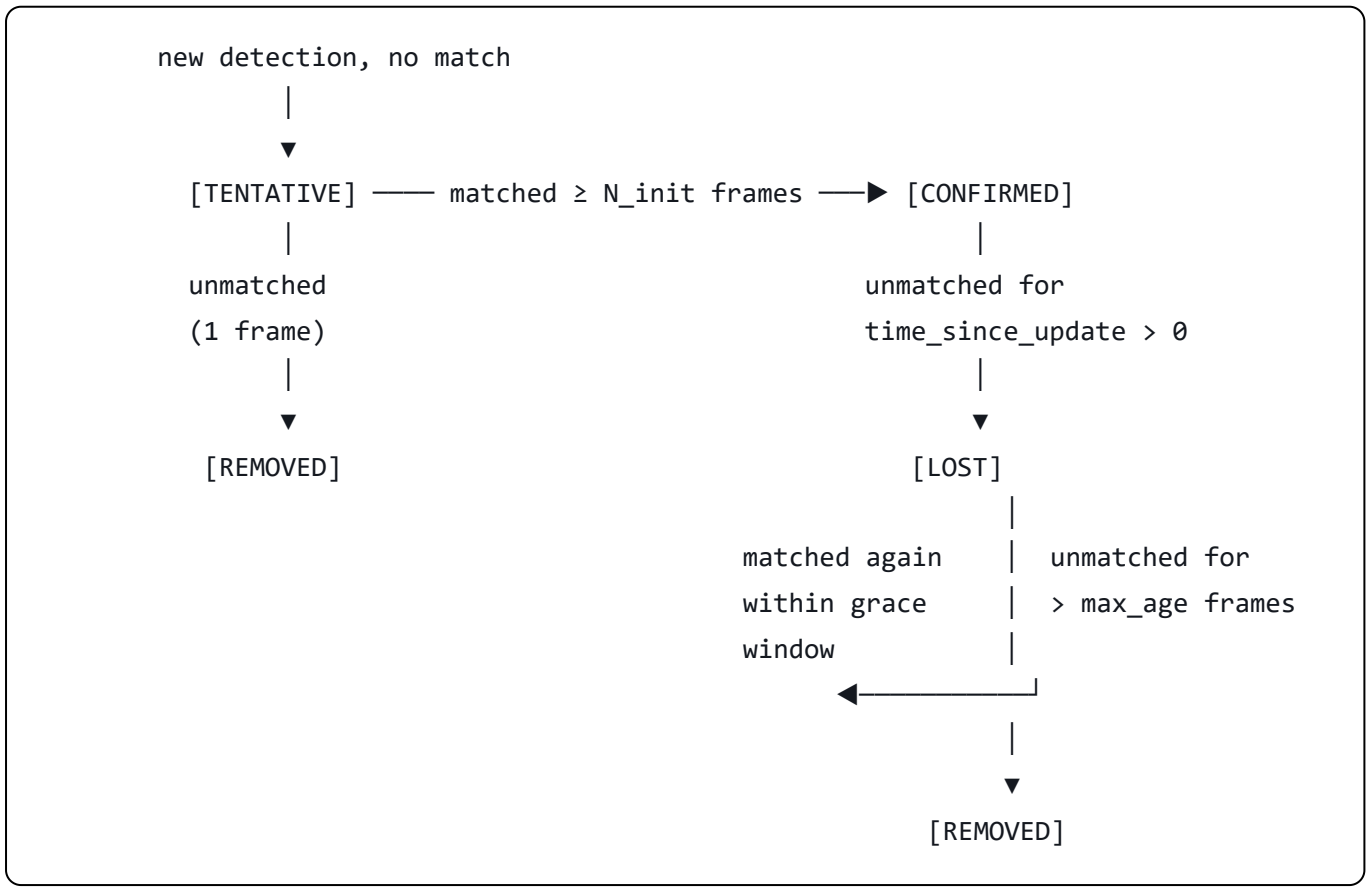
5.3 The Track object

python

```python
class Track:
    """
    One tracked object across time.

    Fields:
        track_id: int                # stable identity
        state: str                   # "tentative" | "confirmed" | "lost" | "removed"
        kalman_state: np.ndarray     # [cx, cy, aspect_ratio, height, vx, vy, va, vh]
        history: deque[dict]         # rolling buffer: {frame_id, bbox, trc, score}
        age: int                     # frames since creation
        hits: int                    # consecutive successful matches
        time_since_update: int       # frames since last matched detection
        class_id: int
        reid_embedding: np.ndarray | None   # v2 only
        behaviour: dict               # populated by utils/behaviour.py — see §7
    """
```

Track state machine (standard SORT-family lifecycle, unchanged in principle by TriNetra's additions):



5.4 The novel piece — `trc_gate.py`

This is TriNetra's contribution on top of stock ByteTrack. The idea:

- ByteTrack's stage-2 matching (low-confidence detections) normally uses a **fixed, generous IoU threshold** because it assumes low confidence just means "partially occluded, still trust the box roughly."
- In TriNetra, a low-confidence detection can mean two *different* things:
  1. The object really is occluded/ambiguous (same as standard ByteTrack).
  2. **The dominant modality driving that detection had low TRC** — the box itself is shakier than a normal low-confidence box.
- `trc_gate.py` adjusts the stage-2 matching cost using the detection's TRC value: high TRC → treat the low-confidence box closer to how ByteTrack normally would (still worth matching); low TRC → require a tighter motion consistency (smaller IoU/gating distance) before accepting the match, since the box position itself is less trustworthy.

python

```
def trc_adjusted_cost(iou_cost: float, trc: float, beta: float = 0.5) -> float:
    """
    Scale the stage-2 association cost by TRC.

    - trc close to 1: cost unchanged (trust the detection's box as-is).
    - trc close to 0: cost inflated, making the match harder to accept
      unless motion prediction agrees very closely.

    cost' = iou_cost * (1 + beta * (1 - trc))
    """
    return iou_cost * (1 + beta * (1 - trc))
```

`beta` is a config-exposed tunable (default 0.5), following the same "no hard-coded constants" principle used in Phase 3's TRC weights.

Fallback behaviour: if `trc` is unavailable for a detection (e.g. TRC was disabled upstream), `trc_adjusted_cost` receives `trc=1.0` by default — same "safe default" pattern Phase 3 used (`trc` defaults to full trust when preprocessing is off). This means Phase 6 works even if run against a detector that hasn't wired TRC through yet, just without the adaptive benefit.

---

6. Cross-Modal Re-Identification (Stage 2 / v2 — BoT-SORT hybrid)

Once ByteTrack is validated (§9), the recommended upgrade path adds an appearance Re-ID embedding, but **modality-aware**, not a generic RGB Re-ID network:

python

```
class DualModalReID(nn.Module):
    """
    Produces one appearance embedding per track by combining RGB-crop and
    thermal-crop embeddings, weighted by TRC — same gating principle as
    Phase 4's fusion block.

        embedding = normalize( e_rgb + trc * e_thermal )

    - High TRC: thermal contributes fully to the identity signature —
      useful for re-acquiring a track after RGB-only occlusion (e.g. someone
      walks behind a light-colored wall reflecting glare, but is still warm).
    - Low TRC: thermal is suppressed, embedding falls back to RGB appearance —
      same residual-safety pattern as the Phase 4 TRCGatedFusion block.
    """
```

Why this matters operationally: DeepSORT/BoT-SORT's standard Re-ID model only ever sees RGB crops, so it inherits every RGB failure mode (glare, low light, camouflage-colored

clothing). A thermal-aware Re-ID embedding gives the tracker a second, independent identity signal exactly in the conditions where RGB Re-ID is weakest — which is the same argument that justified TRC in Phase 3 and the fusion gate in Phase 4. This keeps the "adaptive trust" philosophy consistent across all three layers (preprocessing → detection → tracking).

Training requirement: `DualModalReID` needs a small Re-ID training pass (triplet or ArcFace-style loss) on cropped track patches from LLVIP/M3FD sequences (positive pairs = same track ID across frames, negative = different tracks). This is the only trained component Phase 6 introduces — everything else (Kalman filter, Hungarian matching, TRC gating) is deterministic, same "unsupervised where possible" bias as Phase 3.

---

7. Behaviour Analysis (`utils/behaviour.py`)

Once a track has a history buffer of positions over time, compute:

Feature	How it's computed	What it signals
Trajectory smoothness	Variance of frame-to-frame heading angle change	Erratic path (possible evasive/suspicious movement)
Speed profile	Centroid displacement / time, smoothed	Sudden acceleration (running) vs. loitering (near-zero speed)
Dwell time in zone	Frames spent with centroid inside a configured polygon (restricted zone)	Loitering / unauthorized presence
Direction reversal count	Sign changes in velocity vector over a rolling window	Pacing / surveillance-of-the-surveillance behaviour
Path length vs. displacement ratio	Total path length ÷ straight-line distance start→end	High ratio = wandering/searching behaviour
Track-TRC quality	Mean TRC over the track's frames	Confidence flag on the track itself — feeds into Phase 7 as "low-confidence track," not silently hidden

python

```
def compute_behaviour(track: Track, config: dict) -> dict:
    """
    Returns a behaviour dict attached to Track.behaviour, e.g.:
    {
        "avg_speed": 1.4,               # px/frame, smoothed
        "loitering_frames": 220,
        "in_restricted_zone": True,
        "path_efficiency": 0.31,        # displacement / path_length
        "direction_reversals": 4,
        "mean_trc": 0.68,
        "anomaly_flags": ["loitering", "erratic_path"],
    }
    """
```

v1 approach — rule-based thresholds (config-driven, e.g. `loitering_frames > 150`, `path_efficiency < 0.4`): matches the project's existing bias toward starting deterministic/interpretable before layering learning on top (same choice made for TRC in Phase 3).

v2 research direction — unsupervised trajectory modelling: train a small autoencoder or a Gaussian-process motion model on "normal" trajectories per camera/zone; flag high reconstruction error as anomalous. This requires no labels (only "normal" footage) and is a natural extension once enough track history has been logged in deployment.

---

8. Config Additions (`configs/default.yaml`)

yaml

```yaml
tracking:
  algorithm: "bytetrack"          # bytetrack | botsort (v2)
  kalman:
    motion_model: "constant_velocity"
  matching:
    high_conf_threshold: 0.6      # stage-1 matching cutoff
    low_conf_threshold: 0.1       # detections below this are discarded entirely
    iou_threshold_stage1: 0.3
    iou_threshold_stage2: 0.5
  trc_gate:
    enabled: true                 # novel — TriNetra-specific
    beta: 0.5                     # cost inflation strength for low-TRC boxes
  lifecycle:
    n_init: 3                     # frames to confirm a tentative track
    max_age: 30                   # frames a lost track is kept before removal
  reid:                           # v2 only
    enabled: false
    embedding_dim: 128
    weights: "weights/tracker/dual_modal_reid.pt"
  behaviour:
    loitering_frame_threshold: 150
    path_efficiency_threshold: 0.4
    reversal_window: 20
    restricted_zones: []          # list of polygons, per-camera config
```

---

9. Verification & Testing Plan

Following the same rigor pattern as Phase 3's stress tests (deliberately damage input, confirm the metric responds correctly):

9.1 Standard MOT metrics (quantitative)

Evaluate on a held-out LLVIP/M3FD sequence with track-ID ground truth (synthesize simple multi-frame sequences if the datasets don't ship native tracking annotations — this is an open question, see §11):

- **MOTA** (Multi-Object Tracking Accuracy) — combines false positives, false negatives, and ID switches into one score.
- **IDF1** — how well predicted IDs match ground-truth IDs over the whole sequence (better reflects identity consistency than MOTA alone).
- **HOTA** — the modern standard, balances detection and association accuracy separately, recommended as the headline metric.
- **ID switches (IDSW)** — raw count, the number most intuitive to report in a viva.

9.2 TriNetra-specific ablations (this is the actual novelty proof)

Test	What it isolates
ByteTrack, TRC gate **off** (trc≡1)	Baseline tracker performance
ByteTrack, TRC gate **on**	Does TRC-aware matching reduce ID switches on degraded frames?
Track survival across synthetic thermal dropout	Inject frames with forced low TRC (reuse Phase 3's stress-test damage: blur/saturate/flatten) mid-sequence; measure whether tracks survive vs. baseline
Dual-modal Re-ID vs. RGB-only Re-ID (v2)	Does thermal-assisted appearance matching reduce ID switches during RGB occlusion (e.g. simulated glare patch)?

Key behavioural test (mirrors Phase 4's `check_trc_effect`): run the same sequence twice — once with real TRC values, once with `trc` forced to `1.0` everywhere — and confirm the tracks/ID-switch count actually differ on the degraded-frame subset. If they're identical, the gate isn't wired correctly.

9.3 Manual/qualitative verification

- Visualize tracks over a video with `utils/visualize_tracks.py` **[NEW]** — draw persistent-color boxes + trailing path line + track ID + current `mean_trc` badge per track.
- Confirm ID stability by eye across at least one occlusion event and one thermal-crossover event per test clip.

---

10. File Summary

Status	File	Purpose
NEW	`models/tracker/__init__.py`	Package exports
NEW	`models/tracker/kalman_filter.py`	Constant-velocity Kalman filter for bbox state
NEW	`models/tracker/track.py`	Track object + lifecycle state machine
NEW	`models/tracker/matching.py`	IoU cost matrix + Hungarian assignment
NEW	`models/tracker/trc_gate.py`	**TRC-adjusted association cost (novel)**
NEW	`models/tracker/byte_tracker.py`	ByteTrack two-stage association (v1 core)
NEW	`models/tracker/reid_embedder.py`	Dual-modal Re-ID embedding (v2)
NEW	`models/tracker/botsort_tracker.py`	ByteTrack + Re-ID + camera motion comp (v2)
NEW	`utils/behaviour.py`	Trajectory feature extraction + anomaly flags
NEW	`utils/visualize_tracks.py`	Track visualization for QA
MODIFY	`inference/realtime.py`	Wire tracker into live loop
MODIFY	`inference/video.py`	Wire tracker into offline batch processing
MODIFY	`configs/default.yaml`	Add `tracking:` block
MODIFY	Phase 5 inference wrapper	Attach `trc` to each output detection (1-line change)

---

11. Open Questions

> **1. Tracking ground truth.** LLVIP/M3FD are detection datasets, not native multi-object tracking benchmarks with persistent IDs across frames. Do we (a) synthesize short tracked sequences from consecutive video frames where available, (b) hand-annotate a small validation clip set, or (c) evaluate qualitatively first and defer quantitative MOTA/IDF1/HOTA until a purpose-tracked dataset (e.g. a recorded pilot deployment) exists?

> **2. Restricted-zone definition.** Behaviour features like "dwell in zone" need per-camera zone polygons. Is this configured manually per deployment site, or should Phase 6 ship a simple annotation tool for drawing zones on a reference frame?

> **3. Re-ID training data.** `DualModalReID` (§6) needs positive/negative track-patch pairs. Do we generate these from tracked LLVIP sequences once ByteTrack v1 is running (self-bootstrapping), or hold this off until a labelled Re-ID dataset is available?

> **4. Scalar vs. spatial TRC in the gate.** Phase 3 already wires a spatial `trc_map` (currently off by default). Should `trc_gate.py` eventually use the region under each bounding box from `trc_map` instead of the frame-level scalar TRC, for more precise per-detection trust? Recommended: ship scalar first (matches Phase 3/4's own "scalar first" precedent), revisit if ablations show it matters.

---

12. Quick Reference — Likely Review Questions

Q: Why ByteTrack and not DeepSORT as the starting point? A: ByteTrack needs no trained Re-ID model, is faster (important for Phase 9's real-time goal), and its low-confidence-detection recovery step is a natural hook for TRC-aware matching — a low-confidence box with high TRC is exactly the case worth rescuing.

Q: What's actually novel about Phase 6, versus just using ByteTrack off-the-shelf? A: The TRC gate (`trc_gate.py`) — it makes the *matching cost itself*, not just the detection, aware of per-frame sensor reliability from Phase 3. This extends TriNetra's "adaptive trust" principle from preprocessing (Phase 3) and fusion (Phase 4) into tracking, rather than treating tracking as a bolt-on afterthought.

Q: Does tracking need training? A: The core tracker (Kalman filter + Hungarian matching + TRC gate) needs **no training** — same unsupervised, deterministic bias as Phase 3. Only the v2 upgrade, dual-modal Re-ID embeddings, requires a training pass, and even that only touches a small embedding network, not the tracker's core logic.

Q: How do you evaluate a tracker? A: Standard MOT metrics — MOTA, IDF1, and HOTA — plus a TriNetra-specific ablation: TRC gate on vs. off, measured specifically on synthetically degraded (blurred/saturated/flattened) thermal segments reusing Phase 3's own stress-test methodology.

Q: What happens if a track loses TRC support entirely (sensor dropout)? A: The TRC gate's `trc=1.0` fallback default (§5.4) means the tracker degrades gracefully to standard motion-only ByteTrack behaviour — it doesn't crash or refuse to track, it just loses the adaptive advantage. This mirrors the same "graceful degradation" principle planned for Phase 9 deployment.

Pair this document with `PHASE4_IMPLEMENTATION.md` (fusion) and `PHASE3_EXPLAINED.md` (TRC) — Phase 6's novelty (`trc_gate.py`, `DualModalReID`) is a direct extension of the adaptive-trust principle established in those two phases, applied one layer further downstream.